

CURSO DE PYTHON

Capítulo 3

Comentarios y Docstrings en Python

Desde principiante hasta nivel práctico

Enfoque	Desarrollo de Software Python
Nivel	Fundamentos y aplicación práctica
Propósito	Comprender, documentar y mantener código Python

Capítulo técnico, educativo y práctico para estudiar, practicar y repasar.

Contenido del capítulo

1. Objetivo del capítulo
2. Introducción
3. Conceptos previos
4. Comentarios de una línea
5. Comentarios de varias líneas
6. Docstrings
7. Ejemplos progresivos
8. Comparaciones
9. Errores comunes
10. Buenas prácticas
11. Conectando conocimientos
12. Ejercicios
13. Mini proyecto
14. Comprueba tus conocimientos
15. Respuestas
16. Resumen
17. Mapa de aprendizaje

Idea central: el código debe funcionar, pero también debe poder ser entendido, revisado, corregido y mantenido.

1. Objetivo del capítulo

Los comentarios y docstrings permiten explicar la intención del código. Gracias a ellos, un programa puede ser comprendido por su autor, por otros desarrolladores y por el propio autor meses después de haberlo escrito.

Al finalizar este capítulo serás capaz de:

- Identificar comentarios y docstrings.
- Escribir comentarios de una línea con #.
- Documentar módulos, funciones, clases y métodos.
- Consultar documentación con `__doc__` y `help()`.
- Explicar parámetros, retornos, restricciones y unidades.
- Detectar documentación incorrecta o desactualizada.
- Aplicar documentación en sistemas de máquinas, sensores, inventarios y aplicaciones empresariales.
- Crear un pequeño proyecto organizado y documentado.

Criterio de logro

Dominas el tema cuando puedes tomar un programa sencillo, explicar su propósito, documentar sus partes importantes y justificar por qué cada comentario aporta información útil.

2. Introducción

Imagina una máquina industrial con botones, sensores y luces. Si cada elemento está identificado, el operador puede utilizarla con seguridad. Si nada está etiquetado, incluso una máquina que funciona correctamente puede ser difícil de operar.

El código fuente es parecido. Las instrucciones son la maquinaria interna del programa. Los comentarios y docstrings son etiquetas, notas técnicas y manuales breves.

¿Qué es este tema?

Un **comentario** es una anotación que Python no ejecuta como instrucción. Un **docstring** es una cadena de documentación colocada en una posición especial para describir un módulo, una función, una clase o un método.

¿Por qué existe?

- El código puede funcionar y aun así ser difícil de entender.
- Los proyectos cambian y necesitan mantenimiento.
- Varias personas pueden trabajar sobre el mismo programa.
- Una función puede ser utilizada por alguien que no conoce su implementación.
- La documentación permite recordar decisiones y reducir interpretaciones incorrectas.

¿Dónde se utiliza?

- Bibliotecas de software.
- Sistemas de automatización y PLC.
- Inventarios y aplicaciones empresariales.
- Proyectos colaborativos y revisiones de código.
- Herramientas que generan documentación automática.

¿Qué ocurriría si no existiera?

Los programas seguirían ejecutándose, pero sería más difícil comprenderlos, mantenerlos y reutilizarlos. Cada persona tendría que deducir la intención leyendo todas las instrucciones.

Relación con el curso

Los comentarios y docstrings acompañan a variables, tipos de datos, entradas, salidas, operadores, condicionales, ciclos, funciones y clases. Una variable almacena un dato; un comentario puede explicar su unidad. Una función realiza una tarea; un docstring explica cómo utilizarla.

3. Conceptos previos

El **código fuente** es el conjunto de instrucciones escrito por el programador. El **intérprete** es el programa que lee y ejecuta esas instrucciones. La **documentación** es la información que explica cómo funciona o cómo se utiliza una parte del sistema.

```
nombre = "Sensor de temperatura"
print(nombre)
```

Las dos líneas anteriores son ejecutables. En cambio, el siguiente texto es una anotación:

```
# Este texto explica la siguiente instrucción
nombre = "Sensor de temperatura"
```

Python reconoce el símbolo # fuera de una cadena y considera comentario el resto de esa línea.

4. Comentarios de una línea

¿Qué es?

Es una anotación que comienza con #. Todo lo que aparece después, hasta el final de la línea, se considera comentario.

¿Para qué sirve?

Sirve para explicar una decisión, aclarar una unidad, separar secciones o dejar una nota pendiente.

Sintaxis

```
# Comentario independiente
variable = 10 # Comentario al final de la línea
```

Ejemplo básico

```
# Guardamos la cantidad de piezas
piezas = 125

# Mostramos el dato
print(piezas)
```

Explicación línea por línea

- La primera línea describe el dato que se almacenará.
- La segunda línea crea la variable piezas con el valor 125.
- La línea vacía separa visualmente las secciones.
- El segundo comentario explica la finalidad de la salida.
- print(piezas) muestra el valor en la consola.

Resultado esperado

```
125
```

Ejemplo práctico: monitoreo de una máquina

```
# Datos de la máquina
nombre_maquina = "Torno CNC"
modelo = "SE34"
temperatura = 150.5 # Grados Celsius
velocidad = 1500 # Revoluciones por minuto

print("===== MONITOREO DE MÁQUINA =====")
print(f"Máquina: {nombre_maquina}")
print(f"Modelo: {modelo}")
print(f"Temperatura: {temperatura} °C")
```

```
print(f"Velocidad: {velocidad} RPM")
```

El comentario sobre las unidades es importante: un valor numérico sin unidad puede interpretarse de forma incorrecta.

5. Comentarios de varias líneas

Python no tiene una sintaxis especial llamada comentario multilínea. La forma más clara de escribir una explicación extensa es utilizar varias líneas que comiencen con #.

```
# Este bloque explica el proceso.
# Primero se solicita el dato.
# Después se valida.
# Finalmente se muestra un resumen.
```

También pueden aparecer cadenas de texto entre triples comillas, pero no deben confundirse con comentarios.

```
"""
Este texto es una cadena de texto.
No es automáticamente un comentario.
"""
print("Programa en ejecución")
```

Cuando una cadena está colocada como primera instrucción de un módulo, función, clase o método, Python puede reconocerla como docstring. Para notas normales, utiliza #.

6. Docstrings

¿Qué es?

Un docstring es una cadena de texto colocada como primera instrucción de un módulo, función, clase o método. Python la conserva en el atributo especial `__doc__`.

¿Por qué se utiliza?

Permite describir formalmente qué hace un elemento, qué datos recibe, qué devuelve y qué condiciones deben cumplirse para utilizarlo.

Sintaxis general

```
def nombre_funcion(parametro):
    """Explica qué hace la función."""
    return resultado
```

Ejemplo básico

```
def saludar(nombre):
    """Devuelve un saludo personalizado."""
    return f"Hola, {nombre}"

print(saludar("Ana"))
print(saludar.__doc__)
```

Análisis línea por línea

- `def` crea una función llamada `saludar`.
- El docstring describe el propósito de la función.
- `return` devuelve una cadena.
- La llamada ejecuta la función.
- `saludar.__doc__` consulta la documentación almacenada.

Resultado esperado

```
Hola, Ana
Devuelve un saludo personalizado.
```

Docstring profesional de una función

```
def calcular_produccion_total(piezas_turno, numero_turnos):
```

```

"""Calcula la producción total de una máquina.

Parámetros:
    piezas_turno (int): Piezas producidas en un turno.
    numero_turnos (int): Cantidad de turnos.

Retorna:
    int: Total de piezas producidas.

Reglas:
    Los valores deben ser enteros no negativos.
"""
return piezas_turno * numero_turnos

```

El docstring no realiza el cálculo. La operación se ejecuta en return. La documentación explica el contrato de uso: entradas, salida y restricciones.

Docstrings en clases

```

class SensorTemperatura:
    """Representa un sensor que mide temperatura."""

    def __init__(self, nombre, temperatura):
        """Inicializa el sensor."""
        self.nombre = nombre
        self.temperatura = temperatura

```

Docstrings en módulos

```

"""Herramientas para monitorear estaciones de producción.

Contiene funciones para registrar lecturas,
calcular totales y generar reportes.
"""

```

Consultar documentación

```

def convertir(celsius):
    """Convierte Celsius a Fahrenheit."""
    return (celsius * 9 / 5) + 32

print(convertir.__doc__)
help(convertir)

```

La función help() permite explorar objetos y funciones sin leer toda su implementación.

7. Ejemplos progresivos

Nivel 1 — Básico

```

# Inicio del programa
print("Sistema iniciado")

```

El comentario organiza el archivo y no cambia la salida.

Nivel 2 — Intermedio

```

# Datos del empleado
nombre = "Carlos"
horas_trabajadas = 40
pago_por_hora = 85.50

# Calculamos el pago bruto
pago_bruto = horas_trabajadas * pago_por_hora
print(f"Empleado: {nombre}")
print(f"Pago bruto: ${pago_bruto:.2f}")

```

El comentario explica el significado de la operación, no una obviedad.

Nivel 3 — Práctico

```

def mostrar_reporte_maquina(nombre, modelo, temperatura, piezas):
    """Muestra un reporte básico de una máquina industrial.

    nombre (str): Identificador de la máquina.
    modelo (str): Modelo del equipo.
    temperatura (float): Temperatura en °C.

```

```
piezas (int): Piezas producidas en el turno.
"""
print("==== REPORTE DE MÁQUINA =====")
print(f"Nombre: {nombre}")
print(f"Modelo: {modelo}")
print(f"Temperatura: {temperatura} °C")
print(f"Piezas producidas: {piezas}")
```

Nivel 4 — Profesional

```
"""Módulo de reportes para una estación de producción."""

def calcular_eficiencia(realizadas, objetivo):
    """Calcula el porcentaje de cumplimiento.

    Retorna 0.0 si el objetivo es cero.
    """
    if objetivo == 0:
        return 0.0
    return (realizadas / objetivo) * 100

def generar_reporte(estacion, realizadas, objetivo):
    """Genera un resumen de la estación."""
    eficiencia = calcular_eficiencia(realizadas, objetivo)
    # La presentación se concentra en esta función
    return f"Estación: {estacion}\nEficiencia: {eficiencia:.2f}%"
```

En un proyecto mayor, los docstrings explican responsabilidades y los comentarios aclaran decisiones de implementación.

8. Comparaciones y errores de confusión comunes

Concepto	Qué es	Cuándo usarlo	Diferencia
Comentario #	Anotación ignorada como instrucción.	Notas breves y aclaraciones locales.	No se conserva como docstring.
Docstring	Cadena especial de documentación.	Módulos, funciones, clases y métodos.	Se conserva en <code>__doc__</code> .
Cadena normal	Dato de tipo str.	Cuando el programa necesita texto.	Su propósito es representar datos.
Código ejecutable	Instrucciones procesadas por Python.	Para realizar acciones y cálculos.	Cambia el estado o produce resultados.
README	Documento externo del proyecto.	Instalación, uso general y estructura.	No está dentro necesariamente del archivo Python.

Errores de confusión comunes

- No todo texto entre triples comillas es un docstring.
- Un docstring debe estar en la primera instrucción del elemento.
- Un comentario no sustituye una validación real.
- Los nombres claros reducen la necesidad de comentarios.
- Una documentación incorrecta puede ser peor que no documentar.

9. Errores comunes al trabajar con comentarios y docstrings

Usar # dentro de una cadena

```
print("# Esto aparece en pantalla")
```

Qué ocurre: El símbolo está dentro de comillas y forma parte del texto.

Consecuencia: Se imprime literalmente.

Corrección: Usa # fuera de las comillas.

Colocar el docstring después de código

```
def sumar(a, b):
```

```
resultado = a + b
"""Suma dos valores."""
return resultado
```

Qué ocurre: La cadena no es la primera instrucción.

Consecuencia: No se registra como docstring.

Corrección: Colócala inmediatamente después de def.

Contradecir el código

```
# La temperatura está en Fahrenheit
temperatura_celsius = 80
```

Qué ocurre: El comentario afirma una unidad incorrecta.

Consecuencia: Otra persona puede interpretar mal el dato.

Corrección: Corrige el comentario o el nombre.

Comentar lo obvio

```
# Sumamos 1 a contador
contador = contador + 1
```

Qué ocurre: No aporta contexto.

Consecuencia: Aumenta el ruido visual.

Corrección: Comenta la intención o la razón.

Dejar TODO sin contexto

```
# TODO: arreglar esto
```

Qué ocurre: No explica qué falla ni qué se espera.

Consecuencia: La tarea puede quedar olvidada.

Corrección: Escribe una nota accionable.

Confundir comentario con validación

```
# El sensor siempre entrega un valor válido
lectura = input("Lectura: ")
```

Qué ocurre: El comentario describe una suposición.

Consecuencia: El programa puede recibir datos inválidos.

Corrección: Implementa validaciones reales.

10. Buenas prácticas

Comenta el porqué. El código suele mostrar qué hace; el comentario debe explicar la razón, la regla o la decisión.

Usa nombres descriptivos. Un nombre claro comunica significado y reduce comentarios innecesarios.

Mantén la documentación actualizada. La documentación antigua puede orientar al lector hacia una conclusión incorrecta.

Coloca cada comentario cerca de su código. La proximidad evita confusiones.

Documenta interfaces públicas. Las funciones y clases reutilizables necesitan explicar entradas, salidas y condiciones.

Indica unidades y restricciones. En sistemas industriales, °C, RPM, kilogramos y límites son información crítica.

No ocultes código confuso. Si una sección requiere demasiada explicación, quizá deba dividirse en funciones.

No incluyas información sensible. Nunca escribas contraseñas, tokens o claves privadas en comentarios.

Usa un formato consistente. La consistencia facilita leer y procesar documentación automáticamente.

Revisa durante los cambios. Cada modificación importante debe comprobar que la documentación sigue siendo correcta.

11. Conectando conocimientos

Conocimiento	Conexión
Variables	Explicar significado, unidad y restricciones.
Tipos de datos	Indicar si se espera str, int, float, bool u otro tipo.
Entradas y salidas	Documentar qué se solicita y qué se muestra.
Operadores y matemáticas	Explicar fórmulas y reglas de cálculo.
Condicionales	Aclarar reglas de negocio y límites.
Ciclos	Describir qué datos se recorren y cuándo termina el proceso.
Funciones	Documentar parámetros, retorno, errores y responsabilidad.
Clases	Explicar qué representa la entidad y qué hacen sus métodos.

Ejemplo integrado

```
def clasificar_temperatura(temperatura_celsius):
    """Clasifica una lectura de temperatura industrial."""
    # Los límites son reglas de operación de ejemplo
    if temperatura_celsius < 60:
        return "Normal"
    elif temperatura_celsius <= 90:
        return "Advertencia"
    else:
        return "Crítica"

lectura = float(input("Temperatura en °C: "))
estado = clasificar_temperatura(lectura)
print(f"Estado: {estado}")
```

Aquí se integran entrada, conversión, función, condicionales, salida y documentación. El docstring explica la responsabilidad; el comentario aclara el origen de los límites.

12. Ejercicios prácticos

No se incluyen soluciones inmediatas. Intenta resolver cada ejercicio, ejecuta tu código y revisa si la documentación coincide con su comportamiento.

Ejercicio 1 — Comprensión básica

Objetivo: Identificar comentarios útiles.

Enunciado: Crea un programa que muestre nombre, modelo y estado de una máquina. Añade comentarios para separar datos y salida.

Requisitos:

- Usa variables descriptivas.
- Incluye al menos dos comentarios útiles.

Pistas:

- No describas cada palabra.
- Explica la estructura del programa.

Ejercicio 2 — Aplicación

Objetivo: Distinguir comentarios de texto ejecutable.

Enunciado: Solicita nombre de empleado, horas trabajadas y pago por hora. Muestra el pago e incluye comentarios sobre unidades y cálculo.

Requisitos:

- Usa input().
- Convierte los datos numéricos.

- Muestra un reporte.

Pistas:

- Comenta el significado del dato.
- Separa captura, cálculo y salida.

Ejercicio 3 — Integración

Objetivo: Crear una función documentada.

Enunciado: Crea una función que reciba piezas producidas y objetivo y devuelva el porcentaje de cumplimiento.

Requisitos:

- Incluye parámetros y retorno en el docstring.
- Evita división por cero.
- Añade un comentario sobre una decisión.

Pistas:

- El docstring va después de def.
- Separa cálculo y explicación.

Ejercicio 4 — Caso práctico

Objetivo: Documentar un sistema industrial.

Enunciado: Registra nombre de sensor, temperatura, presión y estado. Crea una función para mostrar el reporte y documéntala.

Requisitos:

- Indica unidades.
- Usa nombres claros.
- Documenta una regla de estado.

Pistas:

- Piensa como operador del sistema.
- No incluyas información sensible.

Ejercicio 5 — Reto

Objetivo: Organizar un módulo reutilizable.

Enunciado: Diseña funciones para producción total, eficiencia y reporte. Documenta el módulo y cada función pública.

Requisitos:

- Separa cálculos y presentación.
- Prueba casos límite.
- Usa comentarios solo donde aporten contexto.

Pistas:

- Define primero el contrato de cada función.
- Comprueba que el texto coincida con el código.

13. Mini proyecto del capítulo

Nombre: Sistema documentado de monitoreo de una estación de producción

Contexto

Una estación necesita registrar datos de una máquina y generar un reporte legible. El programa será pequeño, pero debe organizarse como el inicio de una aplicación real.

Objetivo

Construir un programa que utilice comentarios y docstrings para explicar datos, funciones, reglas y estructura.

Requisitos funcionales

- Registrar nombre y modelo.
- Registrar temperatura, velocidad y piezas.
- Calcular un indicador de cumplimiento.
- Mostrar un reporte claro.
- Indicar unidades.
- Manejar un caso límite como objetivo cero.

Requisitos técnicos

- Usar nombres descriptivos.
- Incluir comentarios útiles.
- Crear al menos dos funciones.
- Escribir docstrings para las funciones.
- Incluir docstring de módulo.
- Separar entrada, procesamiento y salida.

Paso 1 — Definir los datos

Lista las entradas, resultados, unidades y restricciones.

Paso 2 — Crear el cálculo

Implementa y documenta una función para el indicador.

Paso 3 — Crear el reporte

Crea una función que presente el resumen y comenta decisiones de formato.

Paso final — Revisar

Lee el programa como otra persona y comprueba que la documentación coincide con el código.

Pistas generales

- Define la responsabilidad de cada función en una frase.
- Escribe primero el docstring.
- Implementa después el código.
- Prueba y actualiza la documentación.
- La calidad se mide por claridad, no por cantidad de comentarios.

14. Comprueba tus conocimientos

1. ¿Qué símbolo inicia un comentario de una línea?
2. ¿Qué ocurre con un comentario durante la ejecución?
3. ¿Cuál es la diferencia principal entre comentario y docstring?
4. ¿Dónde se coloca normalmente el docstring de una función?

5. ¿Qué atributo permite consultar documentación?
6. ¿Qué función incorporada muestra ayuda?
7. ¿Por qué no conviene comentar cada instrucción obvia?
8. ¿Qué problema causa un comentario desactualizado?
9. ¿El siguiente fragmento tiene un docstring válido? Explica: `def sumar(a,b): / resultado=a+b / ""Suma"" / return resultado.`
10. ¿Qué información debe incluir un docstring profesional?
11. ¿Por qué son importantes las unidades en comentarios?
12. ¿Qué diferencia existe entre cadena normal y docstring?
13. ¿Por qué un comentario no sustituye una validación?
14. ¿Qué debe explicar preferentemente un comentario?
15. ¿Qué temas del curso se benefician de la documentación?

15. Respuestas y explicaciones

1. El símbolo es #.
2. Python no lo ejecuta como instrucción.
3. El comentario es una anotación general; el docstring es documentación formal accesible mediante `__doc__`.
4. Como primera instrucción después de `def`.
5. El atributo `__doc__`.
6. `help()`.
7. Porque genera ruido visual y dificulta localizar información importante.
8. Puede hacer que el lector entienda mal el programa.
9. No. La cadena no es la primera instrucción de la función.
10. Propósito, parámetros, retorno, restricciones, unidades y excepciones relevantes.
11. Porque un número sin unidad puede interpretarse de varias maneras.
12. Una cadena representa datos; un docstring documenta un elemento en una posición especial.
13. Porque solo describe una intención; la validación debe estar implementada.
14. El porqué, la intención o una regla no evidente.
15. Variables, tipos, entradas, salidas, operadores, condicionales, ciclos, funciones, clases y módulos.

16. Resumen del capítulo

Los comentarios y docstrings hacen que el código sea comprensible, mantenible y reutilizable. # sirve para notas breves; un docstring documenta formalmente módulos, funciones, clases y métodos.

Concepto	Para qué sirve	Cuándo usarlo	Error común
Comentario #	Aclarar una línea o decisión.	Notas locales.	Comentar lo obvio o escribir información incorrecta.
Docstring	Describir una interfaz reutilizable.	Funciones, clases, métodos y módulos.	Colocarlo después de otra instrucción.
<code>__doc__</code>	Consultar documentación.	Explorar objetos.	Suponer que cualquier cadena es docstring.
<code>help()</code>	Mostrar ayuda.	Estudiar o utilizar una API.	No consultar la documentación.
Comentario de unidad	Aclarar magnitudes.	Sensores, máquinas y cálculos.	Confundir °C, °F, RPM u otra unidad.

Ideas clave que debes recordar

- El código debe funcionar y poder entenderse.
- `#` inicia comentarios.
- Los comentarios no sustituyen código ejecutable.
- Los docstrings se conservan en `__doc__`.
- El docstring va inmediatamente después de `def`, `class` o al inicio del módulo.
- Documenta intención, entradas, salidas y restricciones.
- Los nombres claros reducen comentarios innecesarios.
- La documentación debe mantenerse actualizada.
- En sistemas industriales documenta unidades y límites.
- La mejor documentación aporta contexto útil.

17. Mapa de aprendizaje

```
Variables y tipos de datos
↓
Entradas y salidas
↓
Comentarios de una línea
↓
Docstrings en funciones y clases
↓
Código legible y documentado
↓
Funciones reutilizables y proyectos colaborativos
↓
Módulos, pruebas, mantenimiento y documentación profesional
```

Este tema prepara el estudio de funciones, clases, módulos, manejo de errores, pruebas y proyectos más grandes. Cuando el código crezca, la documentación será una herramienta esencial para mantenerlo.

Verificación final

- ¿Qué es? Una forma de explicar el código.
- ¿Por qué existe? Para reducir la dificultad de comprender y mantener programas.
- ¿Para qué sirve? Para comunicar intención, uso y reglas.
- ¿Cómo funciona? `#` marca comentarios y los docstrings se guardan en `__doc__`.
- ¿Cuándo se utiliza? En proyectos que deben entenderse y reutilizarse.
- ¿Cómo se aplica? Documentando datos, funciones, clases y módulos.
- ¿Cuáles son los errores? Comentarios obvios, incorrectos, desactualizados o docstrings mal colocados.
- ¿Cuáles son las buenas prácticas? Claridad, consistencia, unidades y actualización.
- ¿Cómo se practica? Escribiendo programas y revisando su legibilidad.
- ¿Cómo se usa en un proyecto real? Como apoyo para construir sistemas mantenibles.

Fin del capítulo